PlanPilot: Adaptive Academic Scheduling Agent

EECS3311 — Stage 1 Software and Agent Design Report

1. Project Overview

1.1. Problem and Motivation

University students often manage several courses at once, each with different assignments, exams, projects, deadlines, estimated workloads, and levels of importance. Traditional calendar and to-do applications can store these items, but they generally leave the planning decisions to the student. The student must still decide what to work on first, how much time to allocate, how to fit study sessions around classes and other commitments, and what to do when a planned session is missed.

PlanPilot is an adaptive academic scheduling agent designed to help students convert academic tasks and personal availability into a realistic study schedule. The system stores structured information about courses, tasks, deadlines, estimated effort, assessment weight, difficulty, availability, and progress. It then generates a study schedule over a selected planning window, such as the next seven days.

The schedule is not regenerated continuously or automatically without user input. The user generates or regenerates a plan when needed, for example after adding a task, changing availability, or reporting that planned work was completed, partially completed, or skipped. This keeps the user in control while still allowing the agent to adapt to changing circumstances.

1.2. Target Users

The primary target users are university and college students who are balancing multiple courses and deadlines and want help allocating their study time.

A typical user may need to manage:

- assignments
- exams and quizzes
- projects
- readings
- study sessions

- classes
- work shifts

PlanPilot is especially useful when deadlines overlap or when a student has less available time than the total amount of work they would ideally like to complete.

1.3. Proposed Agent

PlanPilot combines deterministic scheduling logic with AI-assisted functionality.

Deterministic components are responsible for:

- storing and validating task information
- calculating remaining task effort
- identifying available time blocks
- detecting schedule conflicts
- calculating workload totals
- tracking progress
- applying prioritization rules
- constructing feasible study blocks

AI-assisted components are responsible for:

- interpreting natural-language task descriptions
- helping explain why a task was prioritized or scheduled at a particular time
- assisting with ambiguous replanning requests
- generating user-friendly explanations of scheduling trade-offs

The purpose of the AI component is not to replace the scheduling engine. Instead, it supports tasks that benefit from natural-language interpretation and reasoning while deterministic software remains responsible for calculations and schedule validity.

1.4. Why an AI Agent Is Appropriate

A normal task manager can store deadlines, but PlanPilot is intended to behave adaptively. The system maintains state about the user's tasks, available time, remaining effort, and progress. It can use this state to generate a plan, receive updated progress, and then produce a revised plan when circumstances change.

For example, if a student planned two hours for an assignment but completed only one hour, the system records the remaining work and can redistribute that work into future available time. If a

new high-priority task is added, the schedule can also be regenerated to account for the new constraint. This creates an agent loop:

User enters tasks and availability → System calculates priorities and free time → System generates a schedule → User reports actual progress → System updates remaining work → System regenerates the schedule when requested

This demonstrates planning, state management, decision-making, and multi-step behavior rather than a simple one-prompt chatbot.

1.5. Planned AI/LLM Integration

The application is planned to use an external Large Language Model through an abstraction such as an LLMClient interface. The initial implementation may use an OpenAI model, but the rest of the application will not depend directly on one specific model provider.

The LLM will be used only for features that require language interpretation or explanation, such as parsing a task description like “prepare for a job interview next Thursday” into structured task data. Core scheduling calculations will remain deterministic and testable.

1.6. User Interfaces

PlanPilot will provide both a graphical user interface and a command-line interface.

The graphical user interface will provide pages such as:

- Dashboard
- Tasks
- Schedule
- Availability
- Progress

The command-line interface will expose the same major application functionality through text-based commands or menu options.

Both interfaces will call the same underlying service layer rather than implementing duplicate scheduling logic.

2. Feature Specifications

2.1. F01 — Course Management

Description:

Allows the user to create, view, edit, and remove courses. Courses are used to group academic tasks when a task belongs to a class. A task is not required to belong to a course, so career, personal, or other non-course tasks can also be scheduled.

GUI Interaction:

The user opens the Courses section and selects an option to add, edit, or remove a course.

Input:

Course code and course name.

Output:

A stored Course record available to other features.

AI Involvement:

Deterministic.

Expected Workflow:

1. The user enters course information.

2. The system validates the input.

3. The course is saved.

4. The GUI or CLI displays the updated course list.

Error/Alternative Cases:

The system rejects blank required fields or invalid course identifiers.

2.2. F02 — Task Management

Description:

Allows the user to create and manage schedulable tasks. Tasks may be academic, career-related, personal, or other. Academic tasks are associated with a course, while non-course tasks such as interview preparation or personal study are represented as general tasks.

GUI Interaction:

The user opens the Tasks section and selects Add Task or edits an existing task.

Input:

Required: task title, deadline, and estimated effort. Optional: course, task category/type, assessment weight, and difficulty. Status is system-managed and defaults to NOT_STARTED.

Output:

A stored Task record.

AI Involvement:

Deterministic.

Expected Workflow:

1. The user enters task information.

2. If the task is academic, the user may optionally associate it with a course.

3. The system validates the information.

4. The task is saved and displayed in the task list.

Error/Alternative Cases:

The system rejects tasks with missing required fields, invalid dates, negative effort, or invalid assessment weight. A course is optional, but if a course identifier is supplied, it must refer to an existing course.

2.3. F03 — Natural-Language Task Entry

Description:

Allows the user to enter a task in ordinary language instead of filling every field manually.

Example:

“OS assignment due next Tuesday, worth 10%, probably around four hours.”

GUI Interaction:

The user enters text into a natural-language task field and selects Parse Task.

Input:

Unstructured task description.

Output:

A proposed structured task containing fields such as title, deadline, weight, and estimated effort.

AI Involvement:

AI/Hybrid.

Expected Workflow:

1. The user enters a natural-language description.

2. The system sends the description to the LLM component.

3. The LLM returns proposed structured fields.

4. The system validates the returned fields.

5. The user reviews and confirms or edits the proposed task.

6. The task is saved only after confirmation.

Error/Alternative Cases:

If required information is missing or the model returns invalid data, the system asks the user to correct or complete the information. The system must not silently save malformed AI output.

2.4. F04 — Availability and Commitment Management

Description:

Allows the user to specify when they are available to study and when fixed commitments make them unavailable.

GUI Interaction:

The user opens the Availability section and adds time blocks for classes, work, unavailable periods, or free study time.

Input:

Date/day, start time, end time, and availability/commitment type.

Output:

Stored AvailabilityBlock records used by the scheduling engine.

AI Involvement:

Deterministic.

Expected Workflow:

1. The user enters a time block.

2. The system validates the start and end times.

3. The block is stored.

4. The scheduling engine uses the block when generating future schedules.

Error/Alternative Cases:

The system rejects an end time that is not after the start time and warns about contradictory or overlapping availability data.

2.5. F05 — Task Prioritization

Description:

Calculates the relative priority of incomplete tasks using factors such as deadline proximity, assessment weight, remaining effort, difficulty, and progress.

GUI Interaction:

Priority is normally calculated automatically and displayed in the task list or used during schedule generation.

Input:

Open tasks and their stored attributes.

Output:

A priority value or ordering for tasks.

AI Involvement:

Primarily deterministic.

Expected Workflow:

1. The system retrieves incomplete tasks.

2. A PriorityStrategy evaluates each task.

3. Tasks are ordered based on the selected strategy.

4. The ordering is provided to the scheduling engine.

Error/Alternative Cases:

Tasks with missing or invalid scheduling data are excluded from automatic scheduling until corrected.

2.6. F06 — Study Schedule Generation

Description:

Generates study blocks inside the user's available time for a selected planning window.

GUI Interaction:

The user chooses Generate Schedule and selects or accepts a planning period, with the default being approximately the next seven days.

Input:

Tasks, priorities, remaining effort, deadlines, availability, and scheduling preferences.

Output:

A Schedule containing StudyBlock objects.

AI Involvement:

Hybrid. Deterministic logic constructs a valid schedule; AI may assist with contextual trade-offs or explanations.

Expected Workflow:

1. The user requests a schedule.

2. The system retrieves open tasks and available time.

3. Tasks are prioritized.

4. The planner allocates study blocks without overlapping fixed commitments.

5. The schedule is validated.

6. The schedule is displayed to the user.

Error/Alternative Cases:

If all required work cannot fit within available time, the system reports the deficit rather than pretending the schedule is feasible.

2.7. F07 — Schedule Conflict and Workload Recovery

Description:

Detects overlapping schedule blocks and situations where the total required effort exceeds the user's available time. If a valid schedule cannot be generated, the system identifies the cause and suggests possible adjustments while leaving the final decision to the user.

GUI Interaction:

Warnings and recovery suggestions appear on the dashboard or schedule view. The user can review the problem and choose whether to adjust availability, extend the planning window, defer a lower-priority task, or keep the current constraints.

Input:

Study blocks, fixed commitments, task workload, task priorities, deadlines, and available time.

Output:

Conflict warnings, workload-deficit information, the reason the schedule is infeasible, and possible recovery options.

AI Involvement:

Hybrid. Conflict and workload calculations are deterministic. AI may help explain the conflict and generate user-friendly recovery suggestions based on the calculated scheduling state.

Expected Workflow:

1. The system compares generated study blocks with each other and with fixed commitments.
2. The system totals the remaining required effort.
3. The system compares required effort with available time.
4. If the schedule is feasible, generation continues normally.
5. If the schedule is infeasible, the system calculates the workload deficit and identifies the main constraints causing the problem.
6. The system generates possible recovery options, such as extending the planning window, changing availability, or moving a lower-priority task outside the current window.
7. The user reviews the suggestions and chooses whether to apply one.
8. If the user accepts a change, the system replans using the updated constraints.

Error/Alternative Cases:

If no valid schedule can be produced under the current constraints, the system does not silently drop tasks or modify deadlines. It displays the conflict and possible alternatives, but the user must approve any meaningful scheduling change.

2.8. F08 — Progress Tracking

Description:

Tracks task progress using both scheduled study blocks and user corrections. When a scheduled study block passes, PlanPilot may infer that the planned work was completed and update the task's progress automatically. The user can correct this assumption by marking the work as completed, partially completed, or skipped.

GUI Interaction:

Progress is displayed in the task list and schedule view. Users can adjust inferred progress through actions such as Completed, Partially Completed, or Skipped when the actual outcome differs from the plan.

Input:

Scheduled study blocks, task estimated effort, existing progress, and optional user-reported corrections.

Output:

Updated completed effort, remaining effort, task status, and an indication of whether the current schedule may need replanning.

AI Involvement:

Deterministic. Progress updates are based on scheduled block duration, stored task effort, and explicit user corrections. No LLM is required for the core progress calculation.

Expected Workflow:

1. A scheduled study block reaches its end time.
2. The system tentatively counts the planned block duration toward the task's completed effort.
3. The remaining effort is recalculated.
4. If the task's required effort has been satisfied, the task may be marked COMPLETED.

5. If the user actually completed less work, skipped the block, or finished more than expected, the user can correct the recorded progress.
6. The system recalculates completed and remaining effort using the corrected information.
7. If the change affects the remaining schedule, PlanPilot marks the schedule as needing replanning.

Error/Alternative Cases:

- If the user marks a block as skipped, its duration is removed from inferred completed effort.
- If the user reports partial completion, only the reported amount is counted.
- Completed effort cannot exceed valid task bounds unless the design explicitly allows tracking extra time.
- If a correction changes the remaining workload significantly, the system recommends replanning.

Important Constraint:

PlanPilot may infer progress from scheduled study blocks, but the user can override inferred progress at any time. The system should not silently make irreversible assumptions about task completion.

2.9. F09 — Dynamic Rescheduling

Description:

Detects when the current schedule may no longer reflect the user's actual progress, availability, or workload. Instead of automatically regenerating the schedule after every change, PlanPilot summarizes the impact and presents replanning options for the user to choose from.

GUI Interaction:

When enough changes accumulate to make the current plan outdated, the schedule view displays a replanning suggestion with options such as:

- Replan the entire remaining planning window
- Reschedule only incomplete or missed work
- Extend the planning window
- Keep the current schedule

Input:

Updated tasks, remaining effort, current availability, and the selected planning window.

Output:

A revised Schedule.

AI Involvement:

Hybrid. Deterministic logic calculates the changed workload, remaining effort, available time, and conflicts. AI may explain the impact of the changes and recommend suitable replanning options. The user decides whether and how replanning occurs.

Expected Workflow:

1. Progress, task information, or availability changes.
2. The application updates the underlying state.
3. PlanPilot accumulates any unscheduled or changed workload.
4. The existing schedule is marked as potentially outdated.
5. The system summarizes the impact, such as “6 hours of planned work are now incomplete.”
6. PlanPilot presents replanning options.
7. The user selects an option.
8. Only then does SchedulePlanner generate a revised schedule.
9. The new schedule is checked for conflicts and workload feasibility.
10. The revised schedule is displayed.

Error/Alternative Cases:

If no feasible schedule exists, the system shows the workload deficit and does not create overlapping or invalid blocks.

2.10. F10 — Effort Estimation Feedback

Description:

Uses the user’s previous task and progress history to suggest more realistic time estimates for new, similar tasks. The feature helps PlanPilot improve future schedules without requiring the user to manually estimate every task from scratch.

GUI Interaction:

When the user creates a task and sufficient historical data exists, PlanPilot may display a suggested effort estimate. The user can accept the suggestion, edit it, or ignore it.

Input:

New task information, previous similar tasks, scheduled study-block history, recorded progress, and actual time spent when available.

Output:

A recommended effort estimate for the new task.

AI Involvement:

Hybrid. Deterministic logic calculates historical effort patterns, while AI may help determine whether previous tasks are similar enough to use as a reference and generate a user-friendly recommendation.

Expected Workflow:

1. The user creates a new task.
2. PlanPilot checks whether similar completed tasks exist in the user's history.
3. Historical estimated and actual effort are retrieved.
4. The system calculates a typical effort range or adjustment based on the previous tasks.
5. If appropriate, AI helps determine whether the historical tasks are relevant to the new task.
6. PlanPilot presents a suggested effort estimate to the user.
7. The user accepts, modifies, or ignores the suggestion.
8. If accepted, the estimate is used by the scheduling engine when generating future study blocks.

Error/Alternative Cases:

- If there is insufficient history, PlanPilot does not generate a historical estimate and falls back to the user's own estimate or a simple default.
- If previous tasks are not sufficiently similar, they are not used.
- The system does not silently overwrite the user's chosen estimate.
- If the user rejects the recommendation, the manually chosen value is used instead.

2.11. F11 — Schedule Decision Explanation

Description:

Allows the user to ask why a particular task was prioritized or assigned to a particular study block.

GUI Interaction:

The user selects Explain Schedule or asks a question about the current plan.

Input:

Current schedule, task metadata, priority information, and the user's question.

Output:

A natural-language explanation grounded in the actual schedule state.

AI Involvement:

AI.

Expected Workflow:

1. The user requests an explanation.

2. The system collects the relevant deterministic scheduling data.

3. The LLM receives only the relevant structured context.

4. The system displays the explanation.

Error/Alternative Cases:

The explanation must not invent task information that is absent from the stored schedule state.

3. UML Class Diagram

3.1. Design Overview

PlanPilot uses a layered object-oriented design so that the graphical user interface, command-line interface, scheduling logic, application services, and AI integration remain separate. The presentation layer communicates through PlanPilotController, which delegates requests to the appropriate service or AI component. The scheduling subsystem performs deterministic planning

and conflict detection, while AI-specific functionality is isolated behind the LLMClient interface.

A key design decision is that a schedulable Task does not have to belong to a Course. Academic tasks may optionally reference a course, while career, personal, and other tasks can exist independently. Fixed commitments such as a work shift or class meeting are represented as AvailabilityBlock objects because their time is already fixed rather than something PlanPilot needs to schedule.

3.2. Proposed Classes and Interfaces

The main classes and interfaces in PlanPilot are organized into five groups: Presentation, Domain, Scheduling/Services, AI/Agent, and Event/Observer.

A. Presentation Layer

A1. PlanPilotGUI

Purpose: Provides the graphical interface for the user.

Responsibilities:

- Displays courses, tasks, schedules, availability, and warnings.
- Allows the user to create or edit tasks and availability.
- Displays AI-generated suggestions and explanations.
- Sends user actions to PlanPilotController.

Representative Methods:

+ displayTasks()

+ displaySchedule()

+ displayWarning(message)

+ displaySuggestion(message)

A2. PlanPilotCLI

Purpose: Provides command-line access to the same major PlanPilot functionality.

Responsibilities:

- Accepts commands from the user.
- Displays results in text form.
- Sends commands to PlanPilotController.

Representative Methods:

+ start()

+ displayResult(message)

A3. PlanPilotController

Purpose: Acts as the main coordinator between the user interfaces and the rest of the system.

Responsibilities:

- Delegates task operations to TaskService.
- Delegates scheduling operations to ScheduleService.
- Delegates progress updates to ProgressService.
- Sends AI-related requests to PlanningAgent.

Representative Methods:

+ addTask(taskData): Task

+ addCourse(courseData): Course

+ generateSchedule(start, end): Schedule

+ updateProgress(taskId, progress): void

+ requestReplan(option): Schedule

+ requestEffortSuggestion(task): double

+ parseNaturalLanguageTask(text): Task

+ explainSchedule(question): String

B. Domain Classes

B1. Course

Purpose: Represents an academic course.

Attributes:

- id: String

- code: String

- name: String

Representative Methods:

+ rename(name): void

A Course may be associated with multiple AcademicTask objects.

B2. Task <<abstract>>

Purpose: Represents any flexible piece of work that PlanPilot must find time to schedule.

Examples include:

- exam preparation;
- assignments;
- interview preparation;
- personal project work.

Attributes:

- id: String

- title: String

- deadline: LocalDateTime

- estimatedHours: double

- completedHours: double

- difficulty: int

- userImportance: PriorityLevel

- status: TaskStatus

Representative Methods:

+ getRemainingHours(): double

+ recordProgress(hours): void

+ markCompleted(): void

+ markSkipped(): void

B3. AcademicTask

Extends: Task

Purpose: Represents a task associated with an academic course.

Examples:

- EECS3311 Exam 1;
- Database Assignment;
- Operating Systems Project.

Additional Attributes:

- course: Course

- assessmentWeight: double

- taskType: AcademicTaskType

B4. GeneralTask

Extends: Task

Purpose: Represents a schedulable task that does not belong to a course.

Examples:

- prepare for a job interview;
- French practice;
- personal project work.

No additional attributes are required initially.

B5. AvailabilityBlock

Purpose: Represents a period of time that affects when PlanPilot may schedule tasks.

Examples include:

- available study time;
- class;
- work shift;
- appointment;
- interview.

Attributes:

- id: String

- start: LocalDateTime

- end: LocalDateTime

- type: AvailabilityType

Representative Methods:

+ overlaps(other): boolean

+ durationHours(): double

B6. Schedule

Purpose: Represents a generated schedule for a selected planning period.

Attributes:

- id: String

- startDate: LocalDate

- endDate: LocalDate

- studyBlocks: List<StudyBlock>

- status: ScheduleStatus

Representative Methods:

+ addStudyBlock(block): void

+ getStudyBlocks(): List<StudyBlock>

+ markOutdated(): void

A Schedule contains multiple StudyBlock objects.

B7. StudyBlock

Purpose: Represents a specific scheduled period assigned to a task.

Example:

Tuesday, 6:00 PM–8:00 PM

EECS3311 Exam Preparation

Attributes:

- id: String

- task: Task

- start: LocalDateTime

- end: LocalDateTime

- progressStatus: ProgressStatus

Representative Methods:

+ durationHours(): double

+ markCompleted(): void

+ markPartial(): void

+ markSkipped(): void

B8. ProgressRecord

Purpose: Stores information about the user's actual progress.

Attributes:

- id: String

- task: Task

- studyBlock: StudyBlock

- recordedAt: LocalDateTime

- completedHours: double

- status: ProgressStatus

The stored history can later support adaptive effort estimation.

B9. UserPreferences

Purpose: Stores the user's scheduling preferences.

Attributes:

- maxSessionLength: int

- preferredStudyPeriod: StudyPeriod

- breakLength: int

These preferences help SchedulePlanner decide how work should be divided into study blocks.

C. Scheduling and Service Classes

C1. TaskService

Purpose: Manages tasks.

Responsibilities:

- Creates and validates tasks.
- Supports both academic and non-academic tasks.
- Updates and deletes tasks.
- Provides incomplete tasks to the scheduling system.

Representative Methods:

+ createTask(taskData): Task

+ updateTask(task): void

+ getOpenTasks(): List<Task>

+ deleteTask(taskId): void

C2. CourseService

Purpose: Manages course information.

Responsibilities:

- Creates and validates courses.
- Updates and deletes courses.
- Returns stored courses to the controller.

Representative Methods:

+ addCourse(courseData): Course

+ updateCourse(course): void

+ deleteCourse(courseId): void

+ getCourses(): List<Course>

C3. AvailabilityService

Purpose: Manages availability and fixed commitments.

Responsibilities:

- Adds, updates, and removes availability blocks.
- Validates start/end times.
- Detects contradictory availability entries.
- Provides availability data to the scheduler.

Representative Methods:

+ addAvailability(block): void

+ updateAvailability(block): void

+ deleteAvailability(blockId): void

+ getAvailability(start, end): List<AvailabilityBlock>

C4. ScheduleService

Purpose: Coordinates schedule generation and replanning.

Responsibilities:

- Retrieves tasks, availability, and preferences.
- Calls SchedulePlanner.
- Marks an existing schedule as outdated when necessary.
- Handles user-requested replanning.

Representative Methods:

+ generateSchedule(start, end): Schedule

+ replan(option): Schedule

+ markScheduleOutdated(): void

C5. ProgressService

Purpose: Manages progress tracking.

Responsibilities:

- Records completed, partial, or skipped work.
- Updates remaining task effort.
- Corrects automatically inferred progress.
- Determines whether the current schedule should be marked outdated.

Representative Methods:

+ recordProgress(block, status, hours): void

+ inferProgress(block): void

+ updateRemainingEffort(task): void

+ getHistoryForTask(task): List<ProgressRecord>

C6. SchedulePlanner

Purpose: Creates a valid schedule.

Responsibilities:

- Allocates open tasks into available time.
- Splits larger tasks into multiple study blocks.
- Uses task priorities when determining scheduling order.
- Ensures task work occurs before its deadline.

Representative Methods:

+ createSchedule(tasks, availability, preferences): Schedule

+ allocateTask(task): List<StudyBlock>

C7. PriorityStrategy <<interface>>

Purpose: Defines how a task's priority score is calculated.

Method:

+ calculatePriority(task: Task): double

C8. BalancedPriorityStrategy

Implements: PriorityStrategy

Purpose: Calculates priority using several scheduling factors.

Possible factors include:

- deadline proximity;
- remaining effort;
- difficulty;
- assessment weight;
- user-specified importance;
- progress.

Representative Method:

+ calculatePriority(task: Task): double

C9. ConflictDetector

Purpose: Determines whether a schedule is feasible.

Responsibilities:

- Detects overlapping study blocks.
- Detects conflicts with fixed commitments.
- Determines whether the workload exceeds available time.

Representative Methods:

+ hasConflict(block, availability): boolean

+ calculateWorkloadDeficit(tasks, availability): double

+ isScheduleFeasible(schedule): boolean

C10. EffortEstimator

Purpose: Supports adaptive effort estimation.

Responsibilities:

- Reviews previous similar tasks.
- Calculates historical effort patterns.
- Suggests a realistic amount of effort for a new task.

Representative Methods:

+ estimateEffort(task, history): double

+ calculateHistoricalAverage(tasks): double

D. AI and Agent Classes

D1. PlanningAgent

Purpose: Coordinates PlanPilot's AI-assisted functionality.

Responsibilities:

- Interprets natural-language task input.
- Helps generate effort recommendations.
- Suggests recovery options when a schedule is infeasible.
- Suggests replanning options.

- Explains scheduling decisions.

Representative Methods:

+ parseTask(text): ParsedTaskData

+ suggestEffort(task, history): String

+ suggestRecovery(context): List<String>

+ suggestReplanningOptions(context): List<String>

+ explainSchedule(question, context): String

The PlanningAgent does not directly create arbitrary study blocks. The deterministic scheduling classes remain responsible for producing valid schedules.

D2. TaskParser

Purpose: Converts AI-generated task information into structured data.

Responsibilities:

- Parses LLM output.
- Checks that required fields are present.
- Produces structured task information before a Task is created.

Representative Method:

+ parse(text): ParsedTaskData

D3. LLMClient <<interface>>

Purpose: Defines how PlanPilot communicates with an external LLM.

Method:

+ generate(prompt): String

D4. OpenAIAdapter

Implements: LLMClient

Purpose: Handles provider-specific communication with an OpenAI model.

Responsibilities:

- Sends requests to the OpenAI API.
- Receives model responses.
- Hides OpenAI-specific implementation details from the rest of PlanPilot.

Representative Method:

+ generate(prompt): String

E. Event / Observer Components

E1. PlanningObserver <<interface>>

Purpose: Defines how parts of the application are notified when important planning state changes.

Representative Method:

+ onPlanningStateChanged(): void

PlanPilotGUI implements this interface so the interface can refresh when tasks, progress, or schedules change.

E2. PlanningEventPublisher

Purpose: Notifies registered observers when PlanPilot's planning state changes.

Attributes:

- observers: List<PlanningObserver>

Representative Methods:

+ subscribe(observer): void

+ unsubscribe(observer): void

+ notifyObservers(): void

TaskService, ProgressService, and ScheduleService can use this publisher when their operations affect the current planning state.

3.3. Key Relationships and Multiplicities

A. Course and Academic Tasks

Course "1" -------- "0.." AcademicTask*

- One Course can have zero or many AcademicTask objects.
- Each AcademicTask belongs to one Course.
- GeneralTask objects do not belong to a course.

B. Task Inheritance

Task <|-- AcademicTask

Task <|-- GeneralTask

- Task is an abstract superclass.
- AcademicTask and GeneralTask inherit common task attributes and behavior from Task.

C. Schedule and Study Blocks

*Schedule "1" *-------- "0..*" StudyBlock*

- One Schedule contains zero or many StudyBlock objects.
- StudyBlock objects are part of a Schedule, so this is a composition relationship.

D. Task and Study Blocks

Task "1" -------- "0.." StudyBlock*

- One Task may be divided into multiple StudyBlock objects.
- Each StudyBlock is associated with one Task.

E. Task and Progress Records

Task "1" -------- "0.." ProgressRecord*

- One Task may have multiple ProgressRecord objects over time.
- Each ProgressRecord refers to one task.

F. Controller and Services

PlanPilotController --> CourseService

PlanPilotController --> TaskService

PlanPilotController --> AvailabilityService

PlanPilotController --> ScheduleService

PlanPilotController --> ProgressService

PlanPilotController --> PlanningAgent

PlanPilotController --> EffortEstimator

- PlanPilotController coordinates requests from the GUI and CLI.
- It delegates work to the appropriate service or agent component.

G. Scheduling Dependencies

ScheduleService --> SchedulePlanner

SchedulePlanner --> PriorityStrategy

SchedulePlanner --> ConflictDetector

ScheduleService --> TaskService

ScheduleService --> AvailabilityService

BalancedPriorityStrategy ..|> PriorityStrategy

- ScheduleService uses SchedulePlanner to generate or revise schedules.
- SchedulePlanner depends on the PriorityStrategy abstraction for priority calculation.
- BalancedPriorityStrategy implements PriorityStrategy.
- SchedulePlanner uses ConflictDetector to ensure the schedule is feasible.

H. AI / LLM Relationships

PlanningAgent --> LLMClient

TaskParser --> LLMClient

OpenAIAdapter ..|> LLMClient

- PlanningAgent and TaskParser communicate with the AI through the LLMClient interface.
- OpenAIAdapter implements LLMClient.
- This prevents the rest of PlanPilot from depending directly on one specific LLM provider.

I. Observer Relationships

PlanPilotGUI ..|> PlanningObserver

PlanningEventPublisher --> PlanningObserver

TaskService --> PlanningEventPublisher

ProgressService --> PlanningEventPublisher

ScheduleService --> PlanningEventPublisher

- PlanPilotGUI implements PlanningObserver.
- PlanningEventPublisher keeps track of observers and notifies them when planning state changes.
- TaskService, ProgressService, and ScheduleService use the publisher after changes that may affect the current plan.

3.4. AI/Agent Components in the Design

PlanPilot includes dedicated AI/agent components rather than embedding LLM calls directly into the scheduling classes. PlanningAgent coordinates AI-assisted behavior such as natural-language task interpretation, recovery suggestions, adaptive effort recommendations, replanning suggestions, and schedule explanations. LLMClient defines the abstraction used to communicate with an external language model, while OpenAIAdapter provides a concrete implementation for an OpenAI model. TaskParser converts AI-generated task information into structured task data that can be validated before entering the deterministic scheduling system.

3.5. UML Diagram Specification

Figure 1. PlanPilot UML class diagram.

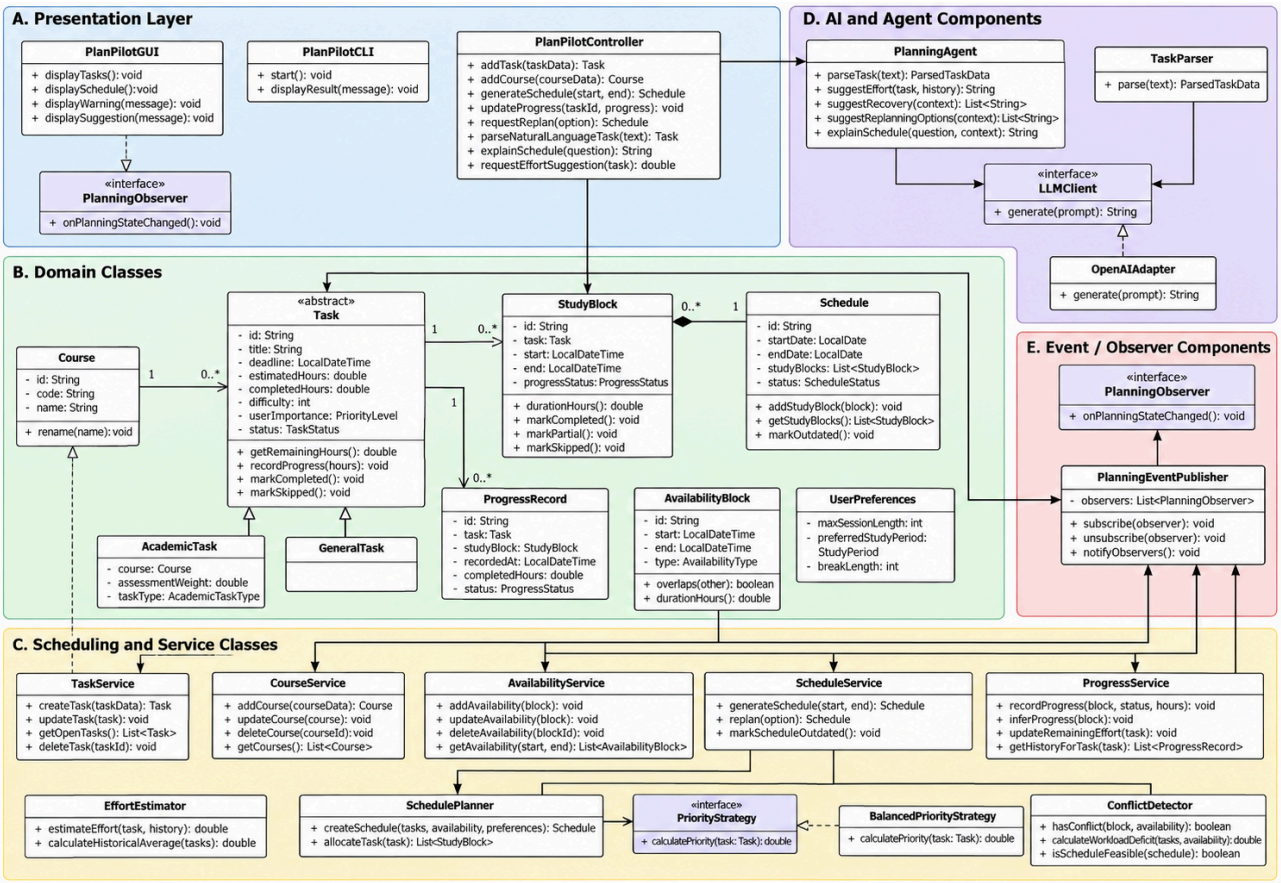


Figure 1. PlanPilot UML class diagram

4. Design Pattern Explanations

PlanPilot uses five design patterns that address specific design problems in the application: Model-View-Controller (MVC), Facade, Strategy, Adapter, and Observer. These patterns help keep the user interface, scheduling logic, AI integration, and application state separated and easier to modify.

4.1. Model-View-Controller (MVC)

Problem:

PlanPilot provides both a graphical user interface and a command-line interface. User-interface code should not contain scheduling calculations, task-management logic, or AI communication.

Participants:

• Model: Course, Task, AcademicTask, GeneralTask, Schedule, StudyBlock, AvailabilityBlock, ProgressRecord, and UserPreferences.

• View: PlanPilotGUI, PlanPilotCLI

• Controllers: PlanPilotController

Roles:

PlanPilotGUI and PlanPilotCLI collect input and display results.

PlanPilotController receives user actions and delegates them to the appropriate service or AI component.

The domain classes represent the application data and state.

Why appropriate:

MVC keeps presentation concerns separate from application logic. For example, clicking Generate Schedule in the GUI does not cause the GUI itself to perform scheduling calculations. Instead, the request goes through PlanPilotController, which delegates the work to ScheduleService.

The CLI can use the same underlying logic without duplicating the implementation.

Without MVC:

Scheduling, validation, progress tracking, and AI calls could become embedded directly in GUI classes. This would create tightly coupled code and make it difficult to reuse the same functionality through the CLI or test the business logic independently

4.2. Facade Pattern

Problem:

The GUI and CLI need access to many different parts of PlanPilot, including task management, progress tracking, scheduling, and AI-assisted functions. The user-interface classes should not need to understand how all these subsystems work together.

Participants:

• Facade: PlanPilotController

• Subsystem classes: TaskService, ScheduleService, ProgressService, and PlanningAgent

Roles:

PlanPilotController provides a simplified entry point to the rest of the application.

For example, it exposes operations such as:

+ addTask(...)

+ generateSchedule(...)

+ updateProgress(...)

+ requestReplan(...)

+ parseNaturalLanguageTask(...)

+ explainSchedule(...)

The controller delegates each operation to the appropriate subsystem.

Why appropriate:

The GUI and CLI interact with one main application-facing class instead of maintaining direct dependencies on several services and AI components. The GUI does not need to know how SchedulePlanner, PriorityStrategy, or ConflictDetector work internally.

Without the Facade:

PlanPilotGUI and PlanPilotCLI would need direct references to many services and agent classes. This would increase coupling and could cause duplicated coordination logic between the GUI and CLI.

4.3. Strategy Pattern

Problem:

There may be different ways to determine which task should receive higher priority when generating a schedule. For example, PlanPilot may consider: deadline proximity, remaining effort, difficulty, and user-specified importance. The scheduling algorithm should not contain one permanently hard-coded priority formula.

Participants:

• Strategy interface: PriorityStrategy

• Concrete strategies: BalancedPriorityStrategy

• Context: SchedulePlanner

Roles:

PriorityStrategy defines:

+ calculatePriority(task: Task): double

BalancedPriorityStrategy implements the calculation using multiple scheduling factors.

SchedulePlanner depends on the PriorityStrategy interface rather than directly depending on BalancedPriorityStrategy.

Why appropriate:

The priority calculation can be changed or extended without changing the main scheduling algorithm. For example, a future implementation could introduce DeadlinePriorityStrategy, which emphasizes upcoming deadlines more heavily. The SchedulePlanner would still call calculatePriority(task) regardless of which strategy is being used.

Without Strategy:

Priority calculations would be embedded directly inside SchedulePlanner. Changing the prioritization algorithm would therefore require modifying the scheduler itself, increasing coupling and making it harder to test different priority approaches independently.

4.4. Adapter Pattern

Problem:

PlanPilot needs to communicate with an external LLM, but the rest of the application should not depend directly on the API format of one specific provider.

For example, an OpenAI API request may use different request objects, authentication, and response formats than another LLM provider.

Participants:

• Target interface: LLMClient

• Adapter: OpenAIAdapter

• Clients: PlanningAgent, TaskParser

Roles:

LLMClient defines a provider-independent operation:

+ generate(prompt: String): String

OpenAIAdapter implements that interface and handles OpenAI-specific API communication.

PlanningAgent and TaskParser depend on LLMClient, rather than depending directly on OpenAIAdapter.

Why appropriate:

The rest of PlanPilot does not need to understand how the selected external LLM provider works.

If the project later changed from OpenAI to another provider, a new adapter could implement LLMClient without requiring changes to the scheduling system or domain classes.

The abstraction also makes testing easier because a fake LLMClient implementation could be substituted during testing.

Without Adapter:

OpenAI-specific API code could appear throughout PlanningAgent, TaskParser, and other application classes. Changing providers would then require modifications across multiple parts of the system.

4.5. Observer Pattern

Problem:

Changes to PlanPilot's state may affect multiple parts of the application. For example, if a user marks a study session as skipped, the remaining task effort changes, the current schedule becomes outdated, and the GUI should display a replanning suggestion. ProgressService should not need to directly manipulate the GUI whenever something changes.

Participants:

• Subject/publisher: PlanningEventPublisher

• Observer interface: PlanningObserver

• Observers: PlanPilotGUI

• Classes Producing Planning Changes: TaskService, ProgressService, and ScheduleService.

Roles:

PlanningObserver defines an update operation such as onPlanningStateChanged(): void.

PlanningEventPublisher maintains registered observers and notifies them when important planning state changes. For example, ProgressService can notify PlanningEventPublisher, which then notifies PlanPilotGUI.

If a user corrects a missed study block, ProgressService updates the underlying task state, and the publisher then notifies the GUI that the displayed schedule or workload information may need to be refreshed.

Why appropriate:

Observer is appropriate because the services that change application state do not need direct references to the GUI. This is particularly useful for progress updates, schedule changes, new or modified tasks, replanning recommendations, and schedule-outdated warnings. It also works well with the F08/F09 design because updating state does not automatically generate a new schedule; instead, the UI can be notified that the current schedule is outdated and offer the user a Replan option.

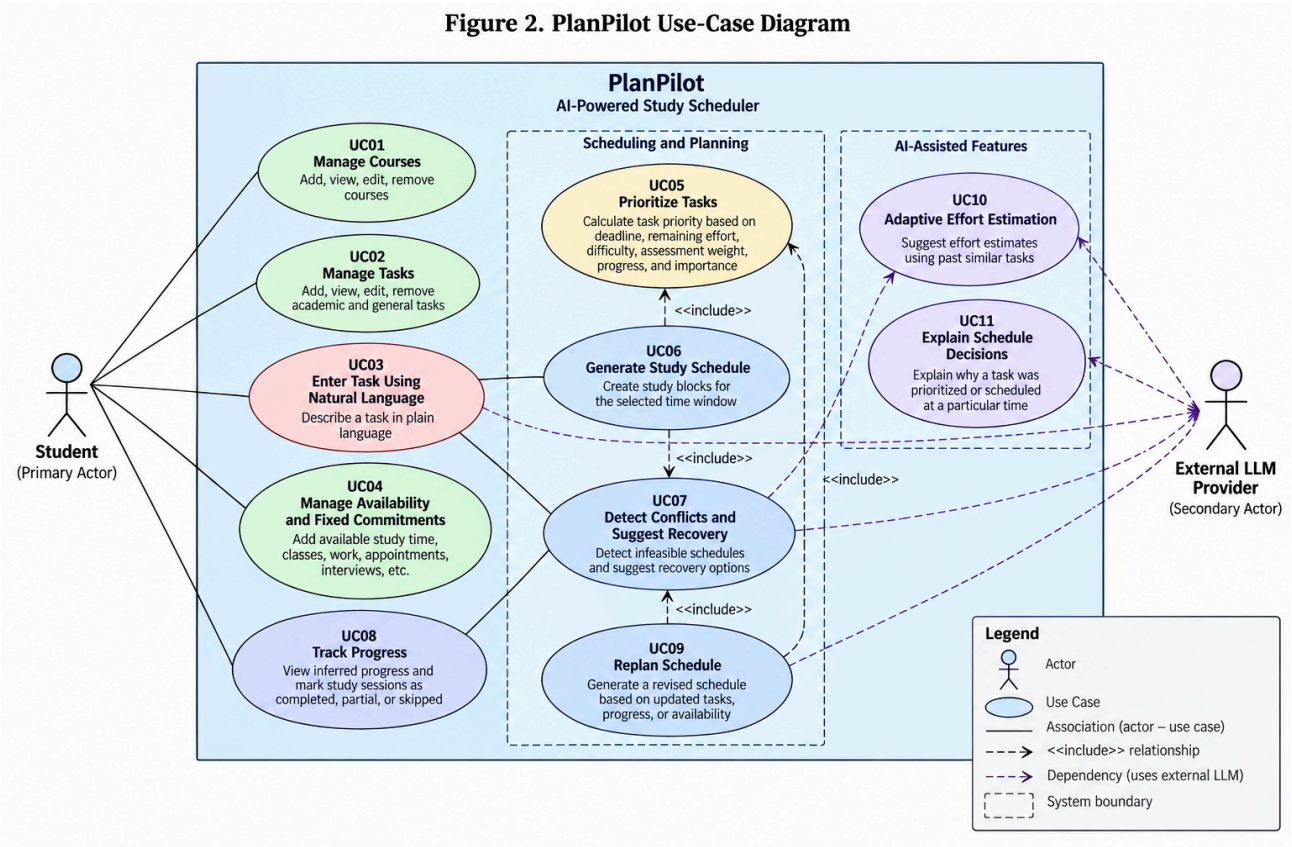
Without Observer:

Services would either need to know directly about GUI classes or the interface would need to manually check every service for changes. This would create unnecessary coupling between application logic and presentation code.

5. Use-Case Diagram

5.1. Use-Case Overview

Figure 2. PlanPilot Use-Case Diagram



5.2. Actors

Student
The main user of PlanPilot. The student creates tasks, manages availability, generates schedules, tracks progress, requests replanning, and asks for AI-assisted suggestions or explanations.

External LLM Provider
Provides language-model capabilities used by PlanPilot for natural-language task parsing, effort suggestions, recovery/replanning suggestions, and schedule explanations.

5.3. Major Use Cases

UC01 — Manage Courses
Create, edit, view, and remove courses.

UC02 — Manage Tasks
Create, edit, view, and remove academic or general tasks.

UC03 — Enter Task Using Natural Language
Describe a task in plain language and let PlanPilot convert it into structured task information.

UC04 — Manage Availability and Fixed Commitments
Add available study time, classes, work shifts, appointments, interviews, and other fixed commitments.

UC05 — Prioritize Tasks
Calculate relative task priority based on deadline, remaining effort, difficulty, assessment weight, progress, and user importance.

UC06 — Generate Study Schedule
Generate study blocks for the selected planning window.

UC07 — Detect Conflicts and Suggest Recovery
Detect overlaps or workload deficits and present recovery options when the schedule is infeasible.

UC08 — Track Progress
Track inferred progress and allow the user to correct completed, partial, or skipped study sessions.

UC09 — Replan Schedule
Present replanning options when the current schedule becomes outdated and generate a revised schedule after the user chooses an option.

UC10 — Adaptive Effort Estimation
Suggest effort estimates for new tasks using previous similar task history.

UC11 — Explain Schedule Decisions
Explain why a task was prioritized or scheduled at a particular time.

5.4. Coverage of Major Features

Feature	Related Use Case
F01 Course Management	UC01 Manage Courses
F02 Task Management	UC02 Manage Tasks
F03 Natural-Language Task Entry	UC03 Enter Task Using Natural Language

F04 Availability / Commitments	UC04 Manage Availability and Fixed Commitments
F05 Task Prioritization	UC05 Prioritize Tasks
F06 Study Schedule Generation	UC06 Generate Study Schedule
F07 Conflict and Workload Recovery	UC07 Detect Conflicts and Suggest Recovery
F08 Progress Tracking	UC08 Track Progress
F09 Assisted Dynamic Replanning	UC09 Replan Schedule
F10 Adaptive Effort Estimation	UC10 Adaptive Effort Estimation
F11 Schedule Decision Explanation	UC11 Explain Schedule Decisions

The use-case model covers all major PlanPilot features defined in Section 2. Features such as task prioritization and conflict detection may also appear as included behavior within schedule generation and replanning.

6. Detailed Use-Case Descriptions

6.1. UC01 — Manage Courses

Primary Actor: Student

Goal:
Create, view, edit, or remove academic courses so academic tasks can be associated with the correct course.

Preconditions:

- PlanPilot is running.
- The student has access to the course-management interface.

Trigger:
The student chooses to add, edit, view, or delete a course.

Main Success Scenario:

1. The student opens the course-management view.
2. The student enters or edits the course code and course name.
3. PlanPilot validates the entered information.
4. The course is created or updated.
5. The updated course list is displayed.

Alternative / Exception Flows:

- If required information is missing, PlanPilot displays a validation message.
- If the student attempts to delete a course that still has associated academic tasks, PlanPilot asks the student to resolve or reassign those tasks first.

Postconditions:
The course information is stored, updated, or removed.

Related Feature:
F01 — Course Management

6.2. UC02 — Manage Tasks

Primary Actor: Student

Goal:
Create, view, edit, or remove schedulable tasks.

Preconditions:

- PlanPilot is running.
- A course may exist if the task is academic, but a course is not required for general tasks.

Trigger:
The student chooses to create, edit, view, or delete a task.

Main Success Scenario:

1. The student opens the task-management view.

2. The student enters the task title and deadline.
3. The student may optionally enter a course, task type, assessment weight, difficulty, estimated effort, and user importance.
4. PlanPilot validates the entered information.
5. If a course is provided, PlanPilot verifies that the course exists.
6. The task is saved.
7. The updated task list is displayed.

Alternative / Exception Flows:

- If required information is missing, the task is not saved.
- Invalid deadlines or numerical values are rejected.
- If a referenced course does not exist, the student is asked to correct the course selection.

Postconditions:
A valid academic or general task is stored.

Related Feature:
F02 — Task Management

6.3. UC03 — Enter Task Using Natural Language

Primary Actor: Student
Secondary Actor: External LLM Provider

Goal:
Allow the student to create a task by describing it in natural language instead of manually entering every field.

Preconditions:

- PlanPilot is running.
- The external LLM service is available.

Trigger:
The student enters a description such as:

> "EECS3311 exam next Wednesday, worth 20%."

Main Success Scenario:

1. The student submits a natural-language task description.
2. PlanningAgent sends the description through LLMClient.

3. The LLM interprets the description.
4. TaskParser converts the response into structured task information.
5. PlanPilot validates the extracted fields.
6. The proposed task information is shown to the student.
7. The student confirms or edits the information.
8. The task is saved.

Alternative / Exception Flows:

- If important information is missing, PlanPilot asks the student to provide it.
- If the LLM returns invalid information, PlanPilot rejects the invalid fields.
- If the LLM service is unavailable, the student can use manual task entry.

Postconditions:
A validated and user-confirmed task is stored.

Related Feature:
F03 — Natural-Language Task Entry

6.4. UC04 — Manage Availability and Fixed Commitments

Primary Actor: Student

Goal:
Define when PlanPilot may schedule flexible work and when the student is unavailable.

Preconditions:
PlanPilot is running.

Trigger:
The student adds or modifies an availability block or fixed commitment.

Main Success Scenario:

1. The student opens the availability view.
2. The student enters a start time, end time, and block type.
3. The block may represent available study time, class, work, an appointment, interview, or another fixed commitment.
4. PlanPilot validates the interval.
5. The availability block is stored.
6. The updated availability is displayed.

Alternative / Exception Flows:

- If the end time occurs before the start time, the entry is rejected.
- If contradictory availability blocks overlap, PlanPilot warns the student.
- The student may edit or remove an existing block.

Postconditions:
The student's availability and fixed commitments are updated.

Related Feature:
F04 — Availability and Commitment Management

6.5. UC05 — Prioritize Tasks

Primary Actor: Student, indirectly through schedule generation or replanning

Goal:
Determine the relative importance of incomplete tasks before they are scheduled.

Preconditions:

- At least one incomplete task exists.
- The task contains enough valid scheduling information.

Trigger:
UC06 Generate Study Schedule or UC09 Replan Schedule requires tasks to be ordered.

Main Success Scenario:

1. PlanPilot retrieves incomplete tasks.
2. SchedulePlanner sends each task to PriorityStrategy.
3. BalancedPriorityStrategy evaluates factors such as:
    - deadline proximity;
    - remaining effort;
    - difficulty;
    - assessment weight;
    - progress;
    - user-specified importance.
4. Each task receives a priority value.
5. Tasks are ordered for scheduling.

Alternative / Exception Flows:

- Tasks with invalid or missing required scheduling information are excluded until corrected.

- If some optional factors are unavailable, priority is calculated using the remaining valid factors.

Postconditions:
The scheduling system has an ordered set of tasks.

Related Feature:
F05 — Task Prioritization

6.6. UC06 — Generate Study Schedule

Primary Actor: Student

Goal:
Generate a valid study schedule for a selected planning period.

Preconditions:

- At least one incomplete task exists.
- Availability information exists.
- Task deadlines are valid.

Trigger:
The student selects Generate Schedule.

Main Success Scenario:

1. The student selects a planning window.
2. ScheduleService retrieves incomplete tasks.
3. PlanPilot retrieves availability and fixed commitments.
4. UC05 Prioritize Tasks is performed.
5. SchedulePlanner allocates task effort into StudyBlock objects.
6. Study blocks are placed before their corresponding deadlines.
7. ConflictDetector checks the proposed schedule.
8. If the schedule is valid, it is saved.
9. The generated schedule is displayed.

Alternative / Exception Flows:

- If there is not enough available time, UC07 is invoked.
- If no usable availability exists, PlanPilot reports that a schedule cannot currently be generated.
- Invalid tasks are excluded and identified to the student.

Postconditions:
A valid schedule is created, or the student receives information explaining why one cannot be created.

Related Feature:
F06 — Study Schedule Generation

6.7. UC07 — Detect Conflicts and Suggest Recovery

Primary Actor: Student
Secondary Actor: External LLM Provider

Goal:
Identify why a schedule is infeasible and present possible recovery options.

Preconditions:
Schedule generation or replanning has detected a conflict or workload deficit.

Trigger:
ConflictDetector determines that the requested workload cannot fit within the current constraints.

Main Success Scenario:

1. ConflictDetector checks for overlapping study blocks.
2. It checks for conflicts with fixed commitments.
3. It compares remaining workload with available time.
4. PlanPilot calculates any workload deficit.
5. PlanPilot identifies the main constraint causing the issue.
6. PlanningAgent may use the LLM to generate understandable recovery suggestions.
7. The student is shown possible actions such as:
    - extending the planning window;
    - changing availability;
    - moving lower-priority work;
    - keeping the existing constraints.
8. The student chooses whether to make a change.

Alternative / Exception Flows:

- If the LLM is unavailable, PlanPilot still displays deterministic conflict and deficit information.
- PlanPilot does not silently delete tasks, change deadlines, or alter important constraints.

Postconditions:
The student understands the scheduling problem and may choose a recovery action.

Related Feature:
F07 — Schedule Conflict and Workload Recovery

6.8. UC08 — Track Progress

Primary Actor: Student

Goal:
Maintain an accurate record of completed, partially completed, or skipped work.

Preconditions:
A scheduled study block or task exists.

Trigger:
A study block finishes, or the student corrects the recorded progress.

Main Success Scenario:

1. A scheduled study block reaches its end time.
2. PlanPilot may tentatively infer that the scheduled work occurred.
3. Completed and remaining task effort are updated.
4. The student may review the recorded progress.
5. If the actual result was different, the student selects:
    - Completed;
    - Partially Completed;
    - Skipped.
6. ProgressService updates the task and creates a ProgressRecord.
7. If the correction affects future scheduled work, the current schedule is marked as outdated.

Alternative / Exception Flows:

- A skipped session contributes no completed effort.
- For partial completion, only the reported amount is counted.
- Invalid progress values are rejected.

Postconditions:
Task progress and remaining effort reflect the latest available information.

Related Feature:
F08 — Progress Tracking and Inferred Completion

6.9. UC09 — Replan Schedule

Primary Actor: Student
Secondary Actor: External LLM Provider

Goal:
Revise the remaining schedule after progress, workload, availability, or task information changes.

Preconditions:

- A current schedule exists.
- Changes have caused the schedule to become outdated or the student manually requests replanning.

Trigger:
PlanPilot recommends replanning, or the student selects **Replan Schedule**.

Main Success Scenario:

1. PlanPilot detects changes such as missed sessions, partial completion, new tasks, deadline changes, or availability changes.
2. The underlying application state is updated.
3. PlanPilot summarizes the effect of the changes.
4. The student is shown replanning options, such as:
    - replan the entire remaining window;
    - reschedule only incomplete work;
    - extend the planning window;
    - keep the current schedule.
5. The student chooses an option.
6. PlanPilot retrieves the latest remaining workload and availability.
7. Task priorities are recalculated.
8. SchedulePlanner generates a revised schedule.
9. ConflictDetector validates the result.
10. The revised schedule is displayed.

Alternative / Exception Flows:

- If the revised workload still cannot fit, UC07 is invoked.
- If the student chooses to keep the current schedule, no new schedule is generated.

- The system does not automatically generate a new schedule after every minor progress update.

Postconditions:
A revised schedule is created only after the student chooses to replan.

Related Feature:
F09 — Assisted Dynamic Replanning

6.10. UC10 — Adaptive Effort Estimation

Primary Actor: Student
Secondary Actor: External LLM Provider

Goal:
Suggest a realistic amount of time for a new task using previous similar task history when available.

Preconditions:
The student is creating or editing a task.

Trigger:
PlanPilot identifies that an effort estimate may be useful for scheduling the task.

Main Success Scenario:

1. The student creates a new task.
2. EffortEstimator retrieves relevant progress history.
3. PlanPilot checks for similar previous tasks.
4. Historical effort patterns are calculated.
5. PlanningAgent may use AI to determine whether past tasks are sufficiently similar to the new task.
6. PlanPilot generates a recommended effort estimate.
7. The recommendation is displayed to the student.
8. The student accepts, edits, or ignores the suggestion.
9. If accepted, the estimate is used during schedule generation.

Alternative / Exception Flows:

- If there is insufficient historical data, AI may suggest an initial estimate from the task information.
- The student must confirm or modify the suggested value before it is used.
- PlanPilot never silently overwrites an estimate chosen by the student.

Postconditions:
The task has an effort estimate approved by the student.

Related Feature:
F10 — Adaptive Effort Estimation

6.11. UC11 — Explain Schedule Decisions

Primary Actor: Student
Secondary Actor: External LLM Provider

Goal:
Explain why PlanPilot prioritized or scheduled a task in a particular way.

Preconditions:

- A generated schedule exists.
- Relevant task and priority information is available.

Trigger:
The student asks a question such as:

> "Why did you schedule my EECS3311 exam preparation before interview prep?"

Main Success Scenario:

1. The student requests an explanation.
2. PlanPilot retrieves the relevant tasks, priorities, deadlines, availability, and schedule information.
3. PlanningAgent constructs context using the actual scheduling data.
4. The information is sent through LLMClient.
5. The LLM produces a natural-language explanation.
6. PlanPilot displays the explanation to the student.

Alternative / Exception Flows:

- If the LLM is unavailable, PlanPilot may display a simpler deterministic explanation using priority factors.
- The explanation does not modify the schedule.
- The LLM is not allowed to invent scheduling facts that are not present in the supplied context.

Postconditions:
The student receives an explanation of the scheduling decision.

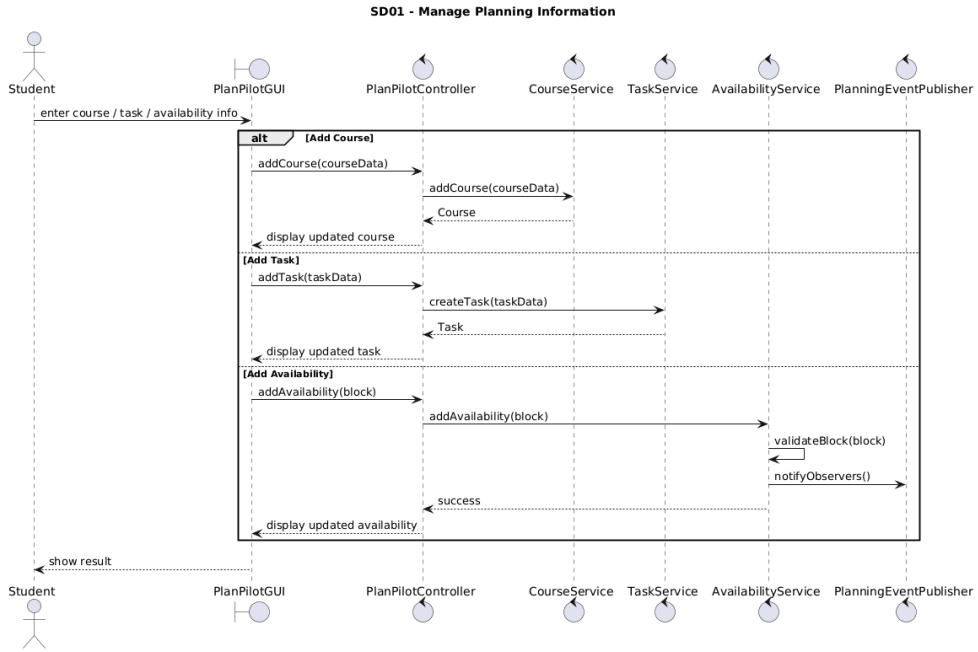
Related Feature:
F11 — Schedule Decision Explanation

7. Sequence Diagrams

The following sequence diagrams show the major interactions between the student, presentation components, controller, application services, domain objects, scheduling components, event/observer components, and AI components.

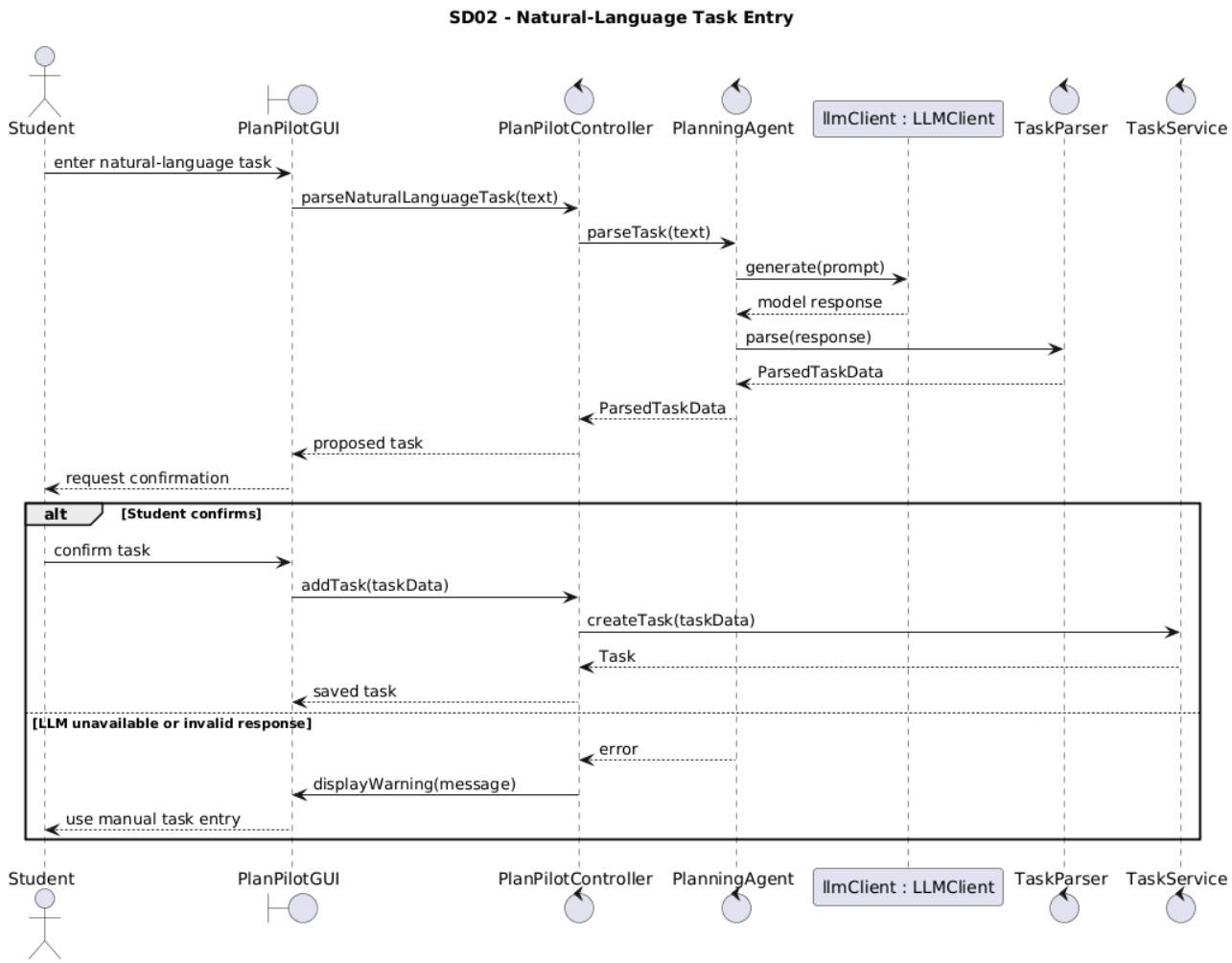
7.1. SD01 — Manage Planning Information

Features covered: F01 Course Management, F02 Task Management, F04 Availability and Commitment Management



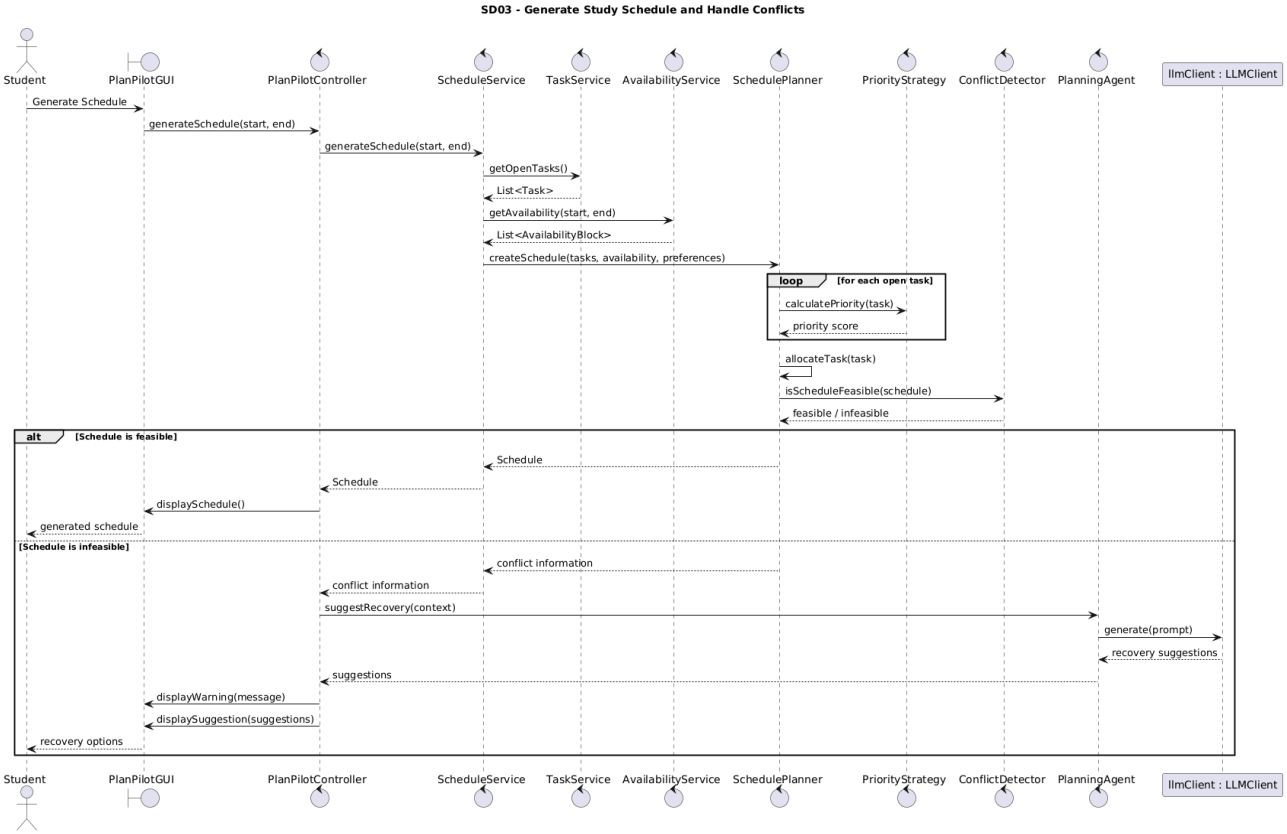
7.2. SD02 — Natural-Language Task Entry

Features covered: F03 Natural-Language Task Entry



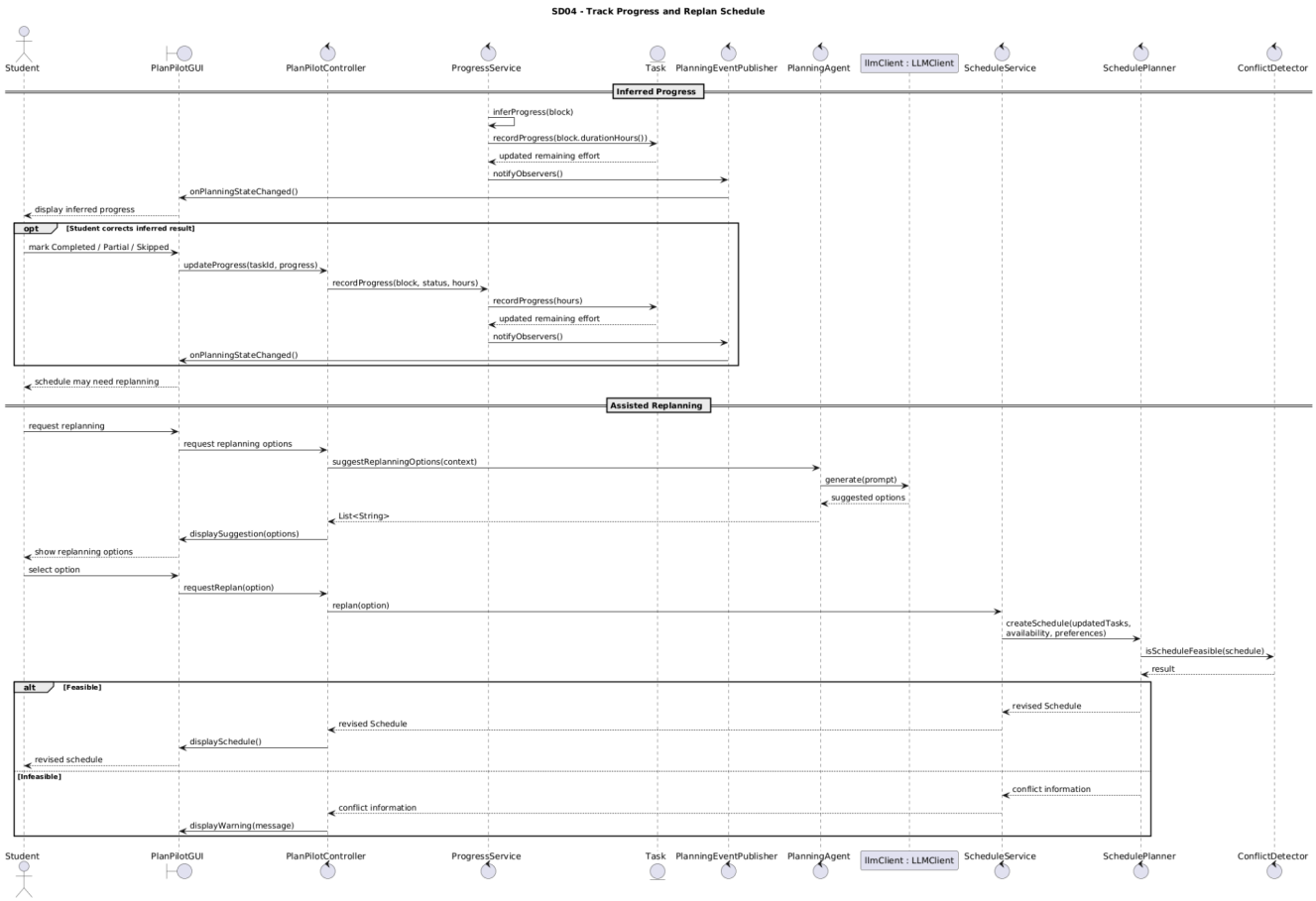
7.3. SD03 — Generate Study Schedule and Handle Conflicts

Features covered: F05 Task Prioritization, F06 Study Schedule Generation, F07 Conflict and Workload Recovery



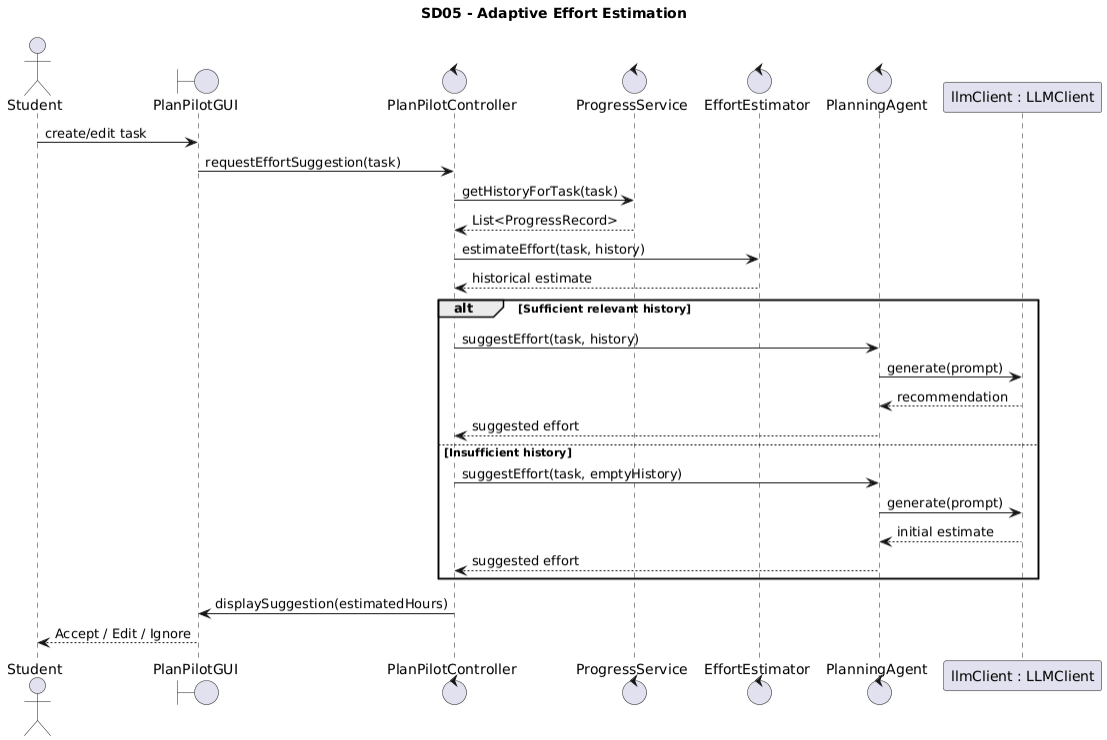
7.4. SD04 — Track Progress and Replan Schedule

Covers: F08 Progress Tracking and Inferred Completion, F09 Assisted Dynamic Replanning



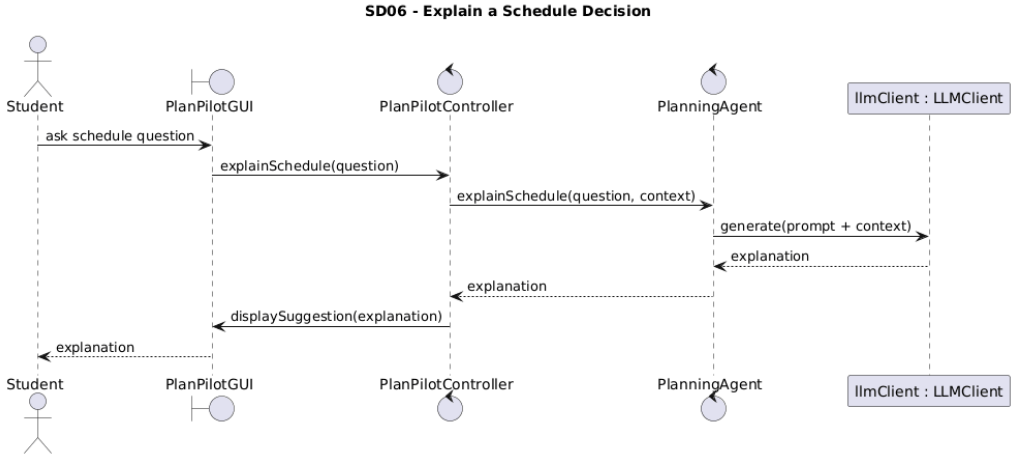
7.5. SD05 — Adaptive Effort Estimation

Covers: F10 Adaptive Effort Estimation



7.6. SD06 — Explain a Schedule Decision

Covers: F11 Schedule Decision Explanation



7.7. Coverage of Important System Interactions

Sequence Diagram	Features Covered

SD01 – Manage Planning Information	F01, F02, F04
SD02 – Natural-Language Task Entry	F03
SD03 – Generate Study Schedule and Handle Conflicts	F05, F06, F07
SD04 – Track Progress and Replan Schedule	F08, F09
SD05 – Adaptive Effort Estimation	F10
SD06 – Explain a Schedule Decision	F11

8. Feature-to-Design Traceability Table

Feature	Related Use Case	Primary Classes / Interfaces	Key Methods	Sequence Diagram	Main Patterns
F01 Course Management	UC01 Manage Courses	Course, CourseService, PlanPilotController	addCourse(), updateCourse(), deleteCourse(), getCourses()	SD01	MVC, Facade
F02 Task Management	UC02 Manage Tasks	Task, AcademicTask, GeneralTask, TaskService, PlanPilotController	createTask(), updateTask(), deleteTask(), getOpenTasks()	SD01	MVC, Facade
F03 Natural-Language Task Entry	UC03 Enter Task Using Natural	PlanningAgent, TaskParser, LLMClient, OpenAIAdapter, TaskService, PlanPilotController	parseNaturalLanguageTask(), parseTask(), generate(), parse(), createTask()	SD02	Adapter, MVC, Facade

	Languag e				
F04 Availability and Commitme nt Manageme nt	UC04 Manage Availabil ity and Fixed Commit ments	AvailabilityBlock, AvailabilityService, PlanPilotController, PlanningEventPubli sher	addAvailability(), updateAvailability(), deleteAvailability(), getAvailability(), notifyObservers()	SD01	MVC, Facade, Observer
F05 Task Prioritizati on	UC05 Prioritize Tasks	PriorityStrategy, BalancedPriorityStr ategy, SchedulePlanner, Task	calculatePriority()	SD03	Strategy
F06 Study Schedule Generation	UC06 Generate Study Schedule	ScheduleService, SchedulePlanner, Schedule, StudyBlock, TaskService, AvailabilityService, ConflictDetector	generateSchedule(), createSchedule(), allocateTask(), isScheduleFeasible()	SD03	Strategy, Facade
F07 Schedule Conflict and Workload Recovery	UC07 Detect Conflicts and Suggest Recovery	ConflictDetector, SchedulePlanner, ScheduleService, PlanningAgent, LLMClient, PlanPilotController	hasConflict(), calculateWorkloadDe ficit(), isScheduleFeasible(), suggestRecovery(), generate()	SD03	Facade, Adapter
F08 Progress Tracking	UC08 Track Progress	ProgressService, Task, StudyBlock, ProgressRecord,	inferProgress(), recordProgress(), updateRemainingEff	SD04	Observer, Facade

and Inferred Completion		PlanningEventPublisher, PlanningObserver	ort(), notifyObservers(), onPlanningStateChanged()		
F09 Assisted Dynamic Replanning	UC09 Replan Schedule	PlanPilotController, PlanningAgent, ScheduleService, SchedulePlanner, ConflictDetector, PriorityStrategy	requestReplan(), suggestReplanningOptions(), replan(), createSchedule(), isScheduleFeasible()	SD04	Facade, Strategy, Adapter, Observer
F10 Adaptive Effort Estimation	UC10 Adaptive Effort Estimation	EffortEstimator, ProgressService, ProgressRecord, PlanningAgent, LLMClient, PlanPilotController	requestEffortSuggestion(), getHistoryForTask(), estimateEffort(), calculateHistoricalAverage(), suggestEffort()	SD05	Facade, Adapter
F11 Schedule Decision Explanation	UC11 Explain Schedule Decisions	PlanPilotController, PlanningAgent, LLMClient, OpenAIAdapter	explainSchedule(), generate(), displaySuggestion()	SD06	Adapter, Facade, MVC

The traceability table shows that every major feature defined in Section 2 is supported by corresponding use cases, classes, methods, and sequence diagrams. The mappings also demonstrate where the selected design patterns are applied. Deterministic scheduling responsibilities remain primarily within ScheduleService, SchedulePlanner, PriorityStrategy, and ConflictDetector, while AI-assisted behavior is isolated through PlanningAgent and the LLMClient abstraction.

9. Feature Realization Explanations

9.1. F01 — Course Management

Course-management requests begin in PlanPilotGUI or PlanPilotCLI and are passed to PlanPilotController. The controller delegates the request to CourseService, which validates the supplied course information and creates, updates, retrieves, or removes Course objects.

Important operations include:

- PlanPilotController.addCourse(...)
- CourseService.addCourse(...)
- CourseService.updateCourse(...)
- CourseService.deleteCourse(...)
- CourseService.getCourses()

A Course can be associated with multiple AcademicTask objects, while GeneralTask objects do not require a course.

Patterns involved: MVC, Facade
AI involvement: None; this feature is deterministic.

9.2. F02 — Task Management

Task-management requests are sent from the GUI or CLI through PlanPilotController to TaskService.

TaskService validates task information and creates either an AcademicTask or GeneralTask. Common information such as title, deadline, estimated effort, completed effort, difficulty, importance, and status is inherited from the abstract Task class.

Important operations include:

- PlanPilotController.addTask(...)
- TaskService.createTask(...)
- TaskService.updateTask(...)
- TaskService.deleteTask(...)
- TaskService.getOpenTasks()

getOpenTasks() is also used by the scheduling subsystem to obtain tasks that still require work.

Patterns involved: MVC, Facade
AI involvement: None for normal manual task management.

9.3. F03 — Natural-Language Task Entry

When the student enters an unstructured task description, PlanPilotController forwards the text to PlanningAgent.

PlanningAgent communicates with the language model through the LLMClient abstraction. At runtime, OpenAIAdapter can provide the concrete implementation of that interface. The returned text is processed by TaskParser, which converts it into structured ParsedTaskData.

The proposed task is returned to the interface for user confirmation. Only after the student confirms or edits the fields does TaskService create the final Task.

The main collaboration is:

PlanPilotGUI

↓

PlanPilotController

↓

PlanningAgent

↓

LLMClient

↓

TaskParser

↓

PlanPilotController

↓

TaskService

This design prevents raw or invalid LLM output from being stored automatically.

Patterns involved: Adapter, Facade, MVC
AI involvement: AI-assisted interpretation; validation and storage remain deterministic.

9.4. F04 — Availability and Commitment Management

Availability requests are sent through PlanPilotController to AvailabilityService.

AvailabilityService creates and manages AvailabilityBlock objects representing either usable study time or fixed commitments such as classes, work shifts, appointments, and interviews.

Important operations include:

- AvailabilityService.addAvailability(...)
- AvailabilityService.updateAvailability(...)
- AvailabilityService.deleteAvailability(...)
- AvailabilityService.getAvailability(...)
- AvailabilityBlock.overlaps(...)

When availability changes may affect an existing plan, AvailabilityService can notify PlanningEventPublisher. Observers such as PlanPilotGUI can then refresh the displayed state or indicate that replanning may be needed.

Patterns involved: MVC, Facade, Observer
AI involvement: None; availability validation is deterministic.

9.5. F05 — Task Prioritization

Task prioritization is performed by the scheduling subsystem rather than directly by the user interface.

SchedulePlanner depends on the PriorityStrategy interface. The current implementation, BalancedPriorityStrategy, calculates a priority score using factors such as:

- deadline proximity;
- remaining effort;
- difficulty;
- assessment weight for academic tasks;
- current progress;
- user-specified importance.

The main interaction is:

SchedulePlanner

↓

PriorityStrategy.calculatePriority(task)

↓

BalancedPriorityStrategy

Because SchedulePlanner depends on the interface rather than one fixed algorithm, another prioritization strategy can later be substituted without changing the scheduler.

Pattern involved: Strategy
AI involvement: None in the core priority calculation.

9.6. F06 — Study Schedule Generation

The student starts schedule generation through the GUI or CLI. PlanPilotController delegates the request to ScheduleService.

ScheduleService retrieves open tasks from TaskService and available time from AvailabilityService. It then passes this information and the user's preferences to SchedulePlanner.

SchedulePlanner:

1. obtains priority values through PriorityStrategy;
2. orders the tasks;
3. divides required work into StudyBlock objects;
4. places those blocks into available time;
5. ensures work occurs before relevant deadlines.

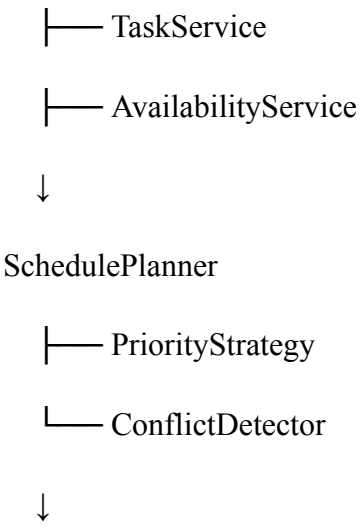
ConflictDetector then verifies that the proposed Schedule is feasible.

The main collaboration is:

PlanPilotController

↓

ScheduleService



Only a valid deterministic schedule is returned to the user.

Patterns involved: Facade, Strategy
AI involvement: The actual schedule construction is deterministic.

9.7. F07 — Schedule Conflict and Workload Recovery

ConflictDetector is responsible for determining whether the proposed workload can be scheduled under the current constraints.

It performs deterministic checks such as:

- hasConflict(...)
- calculateWorkloadDeficit(...)
- isScheduleFeasible(...)

If the proposed schedule is infeasible, SchedulePlanner returns the conflict information through ScheduleService to PlanPilotController.

The controller may then ask PlanningAgent to produce user-friendly recovery suggestions using the already-calculated deterministic information.

For example, suggestions may include:

- extend the planning window;
- increase available study time;
- move lower-priority work;

- keep the existing constraints.

The AI does not silently delete work, change deadlines, or create an invalid schedule. The student chooses whether to apply a change.

Patterns involved: Facade, Adapter
AI involvement: Deterministic conflict detection with AI-assisted explanation and recovery suggestions.

9.8. F08 — Progress Tracking and Inferred Completion

ProgressService manages both inferred progress and user corrections.

When a scheduled StudyBlock reaches its end time, PlanPilot may tentatively infer that the planned work was completed:

ProgressService.inferProgress(block)

The corresponding Task is updated using its progress-related operations, and the remaining effort is recalculated.

If the actual result differs, the student may mark the study block as:

- Completed;
- Partially Completed;
- Skipped.

ProgressService.recordProgress(...) then stores the corrected result in a ProgressRecord and updates the task's remaining work.

After a meaningful change, ProgressService can call PlanningEventPublisher.notifyObservers(). PlanPilotGUI, as a PlanningObserver, can refresh the displayed information and indicate that the schedule may now be outdated.

Importantly, the progress state changes immediately, but a new schedule is not automatically generated.

Patterns involved: Observer, Facade
AI involvement: None; progress calculation is deterministic.

9.9. F09 — Assisted Dynamic Replanning

When changes to progress, tasks, or availability make the current schedule outdated, PlanPilot informs the student rather than automatically replacing the schedule.

PlanningAgent may generate or explain replanning options based on the current deterministic state. Possible choices include:

- replan the entire remaining window;
- reschedule only incomplete work;
- extend the planning window;
- keep the current schedule.

The student chooses an option before a new schedule is generated.

After the selection:

PlanPilotController.requestReplan(option)

↓

ScheduleService.replan(option)

↓

SchedulePlanner.createSchedule(...)

↓

ConflictDetector.isScheduleFeasible(...)

Task priorities are recalculated and remaining work is redistributed using the latest task and availability state.

This preserves the key PlanPilot rule which is: updating state may happen automatically; regenerating the schedule requires a user decision.

Patterns involved: Facade, Strategy, Adapter, Observer
AI involvement: AI may assist with explaining or recommending replanning choices; deterministic components generate and validate the revised schedule.

9.10. F10 — Adaptive Effort Estimation

When an effort suggestion is requested, PlanPilotController retrieves relevant historical progress using ProgressService.getHistoryForTask(task). The history is passed to EffortEstimator.estimateEffort(task, history), which calculates an estimate from previous similar work. PlanningAgent.suggestEffort(...) may use LLMClient to provide an AI-assisted recommendation when useful. The resulting estimate is shown to the student, who may accept, modify, or ignore it. The system never silently replaces a user-selected estimate.

Patterns involved: Facade, Adapter

AI involvement: Hybrid. Historical calculations are deterministic, while AI may assist with the recommendation.

9.11. F11 — Schedule Decision Explanation

When the student asks why a scheduling decision was made, PlanPilotController forwards the question and relevant scheduling context to PlanningAgent.

The context can include deterministic facts such as:

- task deadlines;
- remaining effort;
- priority information;
- difficulty;
- availability;
- actual schedule placement.

PlanningAgent then calls:

LLMClient.generate(...)

to create a natural-language explanation grounded in those facts.

The collaboration is:

Student

↓

PlanPilotGUI

↓

PlanPilotController

↓

PlanningAgent

↓

LLMClient

The returned explanation is displayed to the student through PlanPilotGUI.

The LLM does not change the Schedule, recalculate priorities, or override the deterministic scheduler. Its role is only to explain an already-produced decision.

Patterns involved: Adapter, Facade, MVC
AI involvement: AI-generated explanation grounded in deterministic scheduling state.

9.12. Overall Feature Realization

Across PlanPilot, deterministic software remains responsible for state management, task validation, priority calculation, scheduling, conflict detection, progress tracking, and schedule feasibility. AI functionality is isolated through PlanningAgent and LLMClient and is used where natural-language interpretation, recommendations, or explanations are useful.

This separation allows PlanPilot to provide agent-like adaptive behavior while keeping scheduling decisions testable and preventing the external LLM from directly modifying critical application state.